

DOCUMENTACIÓN DEL SOFTWARE

EcoMarket

Marketplace de productos ecológicos con auditoría de productos

Curso: TALLER DE DESARROLLO DE SOFTWARE

Docente: TORRES CRUZ FRED

Estudiante: JOSEP VLADIMIR GARATE QUISPE

Facultad: Ingeniería Estadística e Informática

Fecha: 04 de August de 2026

Puno — Perú

Índice

Índice	2
1. Visión general	3
2. Arquitectura	4
3. Microservicios (detalle)	5
4. API / Endpoints	6
5. Base de datos	7
6. Seguridad	8
7. Frontend (aplicación web y móvil)	9
8. Despliegue y ejecución	10
9. Roles y flujos	11
10. Manual de uso	12

1. Visión general

1.1 ¿Qué es EcoMarket?

EcoMarket es un marketplace (tienda online) de productos ecológicos y sostenibles, donde cada producto es auditado ingrediente por ingrediente antes de venderse. A diferencia de una tienda convencional, EcoMarket verifica técnicamente que cada producto sea realmente sostenible y le asigna un **Eco-Score** (puntaje ecológico del 0 al 100), dando al comprador la certeza de que lo que adquiere es genuinamente ético y ambiental.

1.2 El problema que resuelve

- **Greenwashing:** muchas marcas se promocionan como ecológicas sin serlo, y el consumidor no puede comprobarlo.
- **Falta de transparencia:** no se conoce el origen ni la composición real de los productos.
- **Desconfianza:** el usuario consciente quiere comprar sostenible pero no sabe en quién confiar.

1.3 Propuesta de valor

- **Auditoría real:** cada producto pasa por un análisis de ingredientes antes de publicarse.
- **Eco-Score visible:** un puntaje claro que resume qué tan ecológico es cada producto.
- **Trazabilidad:** origen, fecha de producción y certificación de cada artículo.
- **Sin greenwashing:** solo se publican productos aprobados por el comité de auditoría.

1.4 Objetivos del sistema

- Ofrecer un catálogo de productos ecológicos verificados.
- Permitir a productores publicar productos que pasan por auditoría.
- Brindar un proceso de compra y pago seguro.
- Garantizar seguridad y trazabilidad en toda la plataforma.

2. Arquitectura

2.1 Estilo: Microservicios

EcoMarket usa **arquitectura de microservicios**: en vez de una sola aplicación monolítica, el sistema se divide en servicios pequeños e independientes, cada uno responsable de una función, que se comunican a través de un **API Gateway**.

Ventajas: cada servicio se desarrolla y despliega por separado; si uno falla no cae todo el sistema; se usa la tecnología más adecuada para cada tarea (Java, Node.js, Python); y se escala de forma independiente.

2.2 Componentes y responsabilidades

Componente	Tecnología	Responsabilidad	Puerto
Frontend Web	Next.js / React	Interfaz de usuario (web, PWA, APK)	3000
API Gateway	Node.js / Kong	Punto de entrada y enrutamiento	8000
auth-service	Java / Spring Boot	Registro, login, seguridad (JWT)	8081
product-service	Java / Spring Boot	Catálogo, pedidos, inventario, auditoría	8082
payment-service	Node.js / Express	Pagos (Stripe, Yape, Plin)	8083
audit-service	Python / FastAPI	Certificación y trazabilidad	8084
ai-engine	Python / FastAPI	Cálculo del Eco-Score	8085
notification-service	Python / FastAPI	Notificaciones al usuario	8086
PostgreSQL	Base de datos SQL	Usuarios, productos, pedidos	5432
MongoDB	Base de datos NoSQL	Registros de auditoría	27017

2.3 Comunicación

- El frontend nunca habla directamente con los microservicios: siempre pasa por el API Gateway.
- El Gateway enruta cada petición según la ruta (/api/auth → auth-service, /api/products → product-service, etc.).
- Algunos servicios se comunican entre sí (payment-service confirma el pedido en product-service y notifica vía notification-service). Estas llamadas internas se protegen con un secreto compartido.

3. Microservicios (detalle)

auth-service — Autenticación (Java / Spring Boot)

Maneja la identidad y seguridad: registro, login, emisión de tokens JWT, cifrado de contraseñas con BCrypt (factor 12), límite de intentos de login y creación del administrador inicial.

product-service — Productos y pedidos (Java / Spring Boot)

Gestiona el catálogo, el inventario/stock, los pedidos y la auditoría (aprobar/rechazar) de productos. Solo muestra públicamente los productos APROBADOS.

payment-service — Pagos (Node.js + Stripe)

Procesa pagos con tarjeta (Stripe) y métodos locales (Yape, Plin, TuPay). Verifica el pago, confirma el pedido y recalcula el monto en el servidor.

audit-service — Auditoría (Python / FastAPI)

Registra el historial de auditoría, genera certificados PDF y analiza ingredientes contra una base de sustancias prohibidas.

ai-engine — Motor de IA (Python / FastAPI)

Calcula el Eco-Score de los productos analizando su información e ingredientes.

notification-service — Notificaciones (Python / FastAPI)

Envía avisos al usuario (por ejemplo, confirmación de pago). Protegido con secreto interno.

API Gateway (Node.js / Kong)

Puerta de entrada única: recibe las peticiones del frontend y las reenvía al microservicio correcto; aplica CORS y límite de peticiones.

4. API / Endpoints

Todas las peticiones pasan por el API Gateway con el prefijo /api/... El sistema expone **26 endpoints**. Acceso: Público (sin login), JWT (autenticado), JWT+rol (rol específico) o Interno (entre servicios).

4.1 auth-service

Método	Ruta	Acceso	Descripción
POST	/api/auth/register	Público	Registrar consumidor
POST	/api/auth/register/producer	Público	Registrar productor
POST	/api/auth/login	Público	Iniciar sesión
GET	/api/auth/me	JWT	Datos del usuario autenticado

4.2 product-service (productos y pedidos)

Método	Ruta	Acceso	Descripción
GET	/api/products	Público	Listar productos aprobados
GET	/api/products/{id}	Público	Detalle de producto
POST	/api/products	Proveedor/Admin	Crear producto
PUT	/api/products/{id}	Proveedor/Admin	Editar producto
POST	/api/products/{id}/audit	Admin	Aprobar/rechazar
POST	/api/orders	JWT	Crear pedido
GET	/api/orders/customer/{id}	JWT (dueño)	Historial de pedidos
POST	/api/orders/{id}/confirm	Interno	Confirmar pago

4.3 payment-service

Método	Ruta	Acceso	Descripción
POST	/api/payments/process-local	JWT	Pago local (Yape/Plin/tarjeta)
POST	/api/payments/create-session	JWT	Sesión de pago Stripe
POST	/api/payments/webhook	Firma	Eventos de Stripe

4.4 audit / ai / notification

Método	Ruta	Acceso	Descripción
POST	/api/audit/{id}/approve	Admin	Aprobar producto
POST	/api/audit/{id}/reject	Admin	Rechazar producto
POST	/api/audit/{id}/generate-certificate	Auth	Certificado PDF
POST	/api/ai/analyze	Interno	Calcular Eco-Score
POST	/api/notifications/push	Interno	Enviar notificación

5. Base de datos

EcoMarket usa dos bases de datos: **PostgreSQL** (relacional) para usuarios, productos y pedidos; y **MongoDB** (NoSQL) para los registros de auditoría.

5.1 Tabla users (usuarios)

Campo	Tipo	Descripción
id	UUID	Identificador único
email	texto (único)	Correo del usuario
full_name	texto	Nombre completo
provider_id	texto	Contraseña cifrada (BCrypt)
role	texto	customer, provider o admin
eco_score	entero	Puntaje ecológico

5.2 Tabla products (productos)

Campo	Tipo	Descripción
id	UUID	Identificador
provider_id	UUID	Proveedor dueño
name / description	texto	Nombre y descripción
price / stock	decimal / entero	Precio y unidades
category	texto	Categoría
eco_score	entero	Puntaje (0-100)
status	texto	PENDING, APPROVED, REJECTED

5.3 Tablas orders / order_items

orders: id, customer_id, total_amount, platform_fee, status (pending/paid), paid_at. **order_items:** order_id, product_id, quantity, unit_price, subtotal.

5.4 MongoDB — colección audits

Guarda el resultado de la auditoría de cada producto: product_id, status, eco_score, badges, audit_hash y details (auditor, observaciones y fecha).

6. Seguridad

6.1 Autenticación

- **Tokens JWT** firmados y con expiración identifican al usuario en cada petición.
- **Contraseñas cifradas** con BCrypt factor 12 (nunca en texto plano).
- El secreto de firma se gestiona por variables de entorno, no en el código.

6.2 Autorización (control de acceso por rol)

- Cada usuario tiene un rol (customer, provider, admin) y solo accede a lo permitido.
- Crear/editar productos: proveedor o admin. Aprobar/rechazar y stock: solo admin.
- La decisión de acceso la toma siempre el backend, no solo el cliente.

6.3 Comunicaciones y datos

- HTTPS/TLS en toda la comunicación.
- Cabeceras de seguridad: CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy.
- CORS restringido a los dominios autorizados.

6.4 Protección contra abuso e integridad de pagos

- Límite de intentos de login (rate limiting) contra fuerza bruta.
- Mensajes de error genéricos (no revelan si un correo existe).
- El monto a cobrar se calcula en el servidor, no se confía en el cliente.
- Verificación de la firma de los webhooks de Stripe y validación de cantidades positivas.

7. Frontend (aplicación web y móvil)

7.1 Tecnología

Next.js 16 + React 19, Tailwind CSS (diseño responsive), Zustand (estado), TanStack Query (consultas) y Axios (HTTP).

7.2 Características

- Responsive: se adapta a celular, tablet y escritorio, con modo claro/oscuro.
- PWA instalable desde el navegador (Android, iPhone, PC).
- APK de Android empaquetada con Capacitor.
- Barra de navegación inferior en móvil (Inicio, Pedidos, Carrito, Cuenta).
- Seguridad en la interfaz: validación de formularios, aviso de Bloq Mayús, bloqueo tras varios intentos.

7.3 Pantallas principales

Pantalla	Descripción
Inicio / Catálogo	Grid de productos con Eco-Score, filtros y buscador
Detalle de producto	Información completa, trazabilidad y compra
Login / Registro	Autenticación con seguridad reforzada
Carrito	Productos seleccionados
Checkout	Pago con Yape, Plin, TuPay o tarjeta
Pedidos / Perfil	Historial de compras y datos de la cuenta
Panel de administración	Gestión de usuarios y auditoría de productos

8. Despliegue y ejecución

8.1 Local con Docker (recomendado)

Con Docker Desktop instalado, en la raíz del proyecto se ejecuta **docker compose up --build**. Esto levanta todo: bases de datos, los 6 microservicios, el gateway y el frontend. Frontend en <http://localhost:3000> y gateway en <http://localhost:8000>.

8.2 En la nube (producción)

Componente	Plataforma
Frontend	Vercel
Backend (microservicios)	Render
PostgreSQL	Neon
MongoDB	MongoDB Atlas

El backend puede desplegarse todo-en-uno (un contenedor con supervisord) o como servicios separados (render.yaml). El stack completo requiere más de 512 MB de RAM para ser estable.

8.3 Generar el APK de Android

- Instalar Android Studio y abrir la carpeta apps/web/android.
- Esperar el Gradle Sync.
- Menú Build → Generate App Bundles or APKs → Generate APKs.
- El APK queda en apps/web/android/app/build/outputs/apk/debug/app-debug.apk.

9. Roles y flujos

9.1 Roles de usuario

Rol	Quién es	Qué puede hacer
Consumidor	Comprador	Explorar, comprar, ver sus pedidos
Productor	Vendedor	Publicar productos (pasan por auditoría)
Administrador	Gestor	Aprobar/rechazar productos, auditar, gestionar

9.2 Flujo de compra (consumidor)

1) Registro/Login → 2) Explorar catálogo → 3) Ver detalle y Eco-Score → 4) Agregar al carrito → 5) Checkout/Pago (Yape, Plin, TuPay o tarjeta) → 6) Pedido confirmado → 7) Ver en 'Mis Pedidos'.

9.3 Flujo de publicación (productor)

1) Registro como productor → 2) Crear producto (estado PENDING) → 3) Análisis de ingredientes y Eco-Score → 4) El administrador aprueba o rechaza → 5) Si se aprueba, aparece en el catálogo público.

9.4 Flujo de auditoría (administrador)

El admin entra al panel, revisa los productos pendientes, analiza la información y aprueba (se publica) o rechaza (indicando motivo). Puede generar un certificado PDF de la auditoría.

10. Manual de uso

10.1 Como consumidor

- Crear cuenta: completar nombre, correo, celular y contraseña segura (mayúscula, minúscula y número); aceptar términos.
- Iniciar sesión con correo y contraseña.
- Explorar el catálogo, filtrar por categoría o buscar.
- Comprar: entrar a un producto → Agregar al carrito → Proceder al pago → elegir método y confirmar.
- Revisar las compras en la sección Pedidos.

10.2 Como administrador

Credenciales por defecto: **admin@ecomarket.pe / 123456789** (cambiar en producción). Desde el panel se pueden ver clientes y productores, revisar productos pendientes, aprobar/rechazar y generar certificados de auditoría.

10.3 Preguntas frecuentes

- **¿No carga al pagar?** El servidor gratuito puede estar 'dormido'; espera ~50 s y reintenta.
- **¿Mi producto no aparece?** Está pendiente de auditoría; solo se muestran los aprobados.
- **¿El pago cobra dinero real?** En modo demostración no; simula el pago pero registra el pedido.

Documentación del software EcoMarket — Universidad Nacional del Altiplano · Facultad de Ingeniería Estadística e Informática.